Cours de Python

Du débutant au mini-projet, avec Termux

Contenu :

- Installation et premiers pas
- Leçon 1 : afficher du texte
- Leçon 2 : variables et calculs
- Leçon 3 : demander une valeur (input, int)
- Leçon 4 : les conditions (if / else)
- Leçon 5 : conditions multiples (elif)
- Leçon 6 : la boucle for
- Leçon 7 : la boucle while
- Leçon 8 : les listes
- Leçon 9 : les fonctions
- Leçon 10 : plus de calculs
- Leçon 11 : le texte (chaînes)
- Leçon 12 : les dictionnaires
- Leçon 13 : les erreurs (try / except)
- Leçon 14 : mini-projet, devine le nombre
- Explication détaillée : print, input et int
- Mémo : les mots à retenir

Installation (Termux)

```
pkg update
pkg install python
python --version
```

Créer un fichier : `nano fichier.py`

- Enregistrer : `CTRL + O`, puis Entrée
- Quitter : `CTRL + X`
- Lancer : `python fichier.py`

Internet sert seulement à installer. Ensuite, Python marche hors ligne.

Leçon 1 : afficher du texte

```
print("Bonjour !")
```

`print()` affiche quelque chose à l'écran.

Leçon 2 : variables et calculs

Une variable est une boîte avec un nom, qui garde une valeur.

```
prenom = "Karim"
age = 20
print(prenom)
print(age)
print("J'ai", age, "ans")
```

```
a = 10
b = 3
print(a + b)   # addition : 13
print(a - b)   # soustraction : 7
print(a * b)   # multiplication : 30
print(a / b)   # division : 3.333...
```

Tout ce qui suit # est un commentaire : Python l'ignore.

Leçon 3 : demander une valeur

```
prenom = input("Comment tu t'appelles ? ")
print("Salut", prenom, "!")
```

`input()` donne toujours du **texte**. Pour calculer, on convertit avec `int()`.

```
a = int(input("Premier nombre : "))
b = int(input("Deuxième nombre : "))
print("La somme est", a + b)
```

Avec du texte, `"5" + "8"` donne `"58"`. Avec des nombres, `5 + 8` donne `13`.

Leçon 4 : les conditions (if / else)

```
age = int(input("Quel âge as-tu ? "))

if age >= 18:
    print("Tu es majeur")
else:
    print("Tu es mineur")
```

- Les `:` sont obligatoires après `if` et `else`.
- La ligne suivante est décalée de 4 espaces (indentation).
- Comparaisons : == égal, `!=` différent, > < >= <=

```
nombre = int(input("Entre un nombre : "))

if nombre > 0:
    print("positif")
else:
    print("négatif ou nul")
```

`else` veut dire « sinon ».

Leçon 5 : conditions multiples (elif)

```
note = int(input("Ta note sur 20 : "))

if note >= 16:
    print("Très bien")
elif note >= 10:
    print("Passable")
else:
    print("Insuffisant")
```

Python lit de haut en bas et s'arrête au premier test vrai. `elif` veut dire « sinon si ».

Exercice : ajoute un niveau « Bien » pour les notes à partir de 14 (un `elif note >= 14:` placé avant celui de 10).

Leçon 6 : la boucle for

```
for i in range(5):
    print("Bonjour", i)
```

- `range(5)` donne 0, 1, 2, 3, 4.
- `range(1, 6)` donne 1, 2, 3, 4, 5.
- Les lignes décalées de 4 espaces sont répétées à chaque tour.

Exercice : affiche les nombres de 1 à 10 avec `for`.

Leçon 7 : la boucle while

`while` répète **tant que** la condition est vraie.

```
compteur = 1

while compteur <= 5:
    print("Compteur :", compteur)
    compteur = compteur + 1

print("Fini !")
```

Attention : sans `compteur = compteur + 1`, la boucle ne s'arrête jamais. Arrête-la avec `CTRL + C`.

Exercice : affiche les nombres de 1 à 10 avec `while`.

Leçon 8 : les listes

Une liste garde plusieurs valeurs dans une seule variable.

```
fruits = ["pomme", "banane", "mangue"]
print(fruits[0])      # pomme (on commence à 0)
print(fruits[2])      # mangue
print(len(fruits))    # 3 (nombre d'éléments)

fruits.append("orange")   # ajouter à la fin
fruits.remove("banane")   # enlever

for fruit in fruits:
    print("J'aime", fruit)
```

Exercice : crée une liste de 5 nombres et affiche leur somme avec une boucle `for`.

Leçon 9 : les fonctions

Une fonction est un bloc de code qu'on écrit une fois et qu'on réutilise. On la crée avec `def`.

```
def dire_bonjour(prenom):
    print("Bonjour", prenom, "!")

dire_bonjour("Karim")
dire_bonjour("Awa")
```

`return` renvoie un résultat :

```
def addition(a, b):
    return a + b

resultat = addition(5, 8)
print(resultat)    # 13
```

Exercice : écris une fonction `carre(n)` qui renvoie n multiplié par n.

Leçon 10 : plus de calculs

```
print(10 // 3)   # 3  (division entière)
print(10 % 3)    # 1  (reste de la division)
print(2 ** 3)    # 8  (puissance : 2 puissance 3)
```

Avec %, on teste si un nombre est pair :

```
n = int(input("Un nombre : "))
if n % 2 == 0:
    print("pair")
else:
    print("impair")
```

Leçon 11 : le texte (chaînes)

```
mot = "Python"
print(len(mot))        # 6 (nombre de lettres)
print(mot.upper())     # PYTHON
print(mot.lower())     # python
print(mot[0])          # P (première lettre)
print(mot.replace("P", "J"))   # Jython

prenom = "Karim"
age = 20
print(f"Je m'appelle {prenom} et j'ai {age} ans")
```

Le `f` devant les guillemets permet de mettre des variables entre `{ }` dans le texte.

Leçon 12 : les dictionnaires

Un dictionnaire range des valeurs avec un nom (une clé), comme une fiche.

```
personne = {"nom": "Karim", "age": 20, "ville": "Abidjan"}
print(personne["nom"])       # Karim
personne["age"] = 21         # modifier
personne["metier"] = "étudiant"   # ajouter

for cle in personne:
    print(cle, ":", personne[cle])
```

Leçon 13 : les erreurs (try / except)

Si l'utilisateur tape du texte au lieu d'un nombre, le programme plante. `try / except` évite ça.

```
try:
    n = int(input("Un nombre : "))
    print("Le double est", n * 2)
except ValueError:
    print("Ce n'est pas un nombre !")
```

Python essaie le bloc `try`. S'il y a une erreur, il fait le bloc `except` au lieu de planter.

Leçon 14 : mini-projet, devine le nombre

Un jeu qui combine tout : `import`, `while`, `input`, `int`, `if / elif / else`.

```
import random

secret = random.randint(1, 20)
essais = 0

print("Devine le nombre entre 1 et 20")

while True:
    try:
        choix = int(input("Ton choix : "))
    except ValueError:
        print("Tape un nombre !")
        continue

    essais = essais + 1

    if choix < secret:
        print("Plus grand !")
    elif choix > secret:
        print("Plus petit !")
    else:
        print("Bravo ! Trouvé en", essais, "essais")
        break
```

`break` sort de la boucle. `continue` repart au début de la boucle. `random.randint(1, 20)` choisit un nombre au hasard.

Exercice : limite le jeu à 5 essais : si l'utilisateur n'a pas trouvé, affiche « Perdu ! » et le nombre secret.

Explication détaillée : print, input et int

print() : afficher
Sert à afficher quelque chose à l'écran.

```
print("Bonjour")   # affiche Bonjour
print(5 + 3)       # affiche 8
print("J'ai", 20, "ans")   # plusieurs choses, séparées par des virgules
```

input() : demander
Le programme s'arrête et attend que l'utilisateur tape quelque chose puis appuie sur Entrée. Ce qui est tapé est gardé dans la variable.

```
prenom = input("Ton prénom ? ")
print("Salut", prenom)
```

int() : convertir en nombre
`input()` donne toujours du **texte**, même si tu tapes 5. Avec du texte, Python ne peut pas calculer. `int()` transforme le texte en vrai nombre entier.

```
a = input("Nombre : ")        # "5" (texte)
b = int(input("Nombre : "))   # 5 (vrai nombre)
```

Avec du texte, `"5" + "8"` donne `"58"` (Python colle les deux). Avec des nombres, `5 + 8` donne `13`.

En résumé :

- `print()` : le programme **parle** à l'utilisateur.
- `input()` : le programme **écoute** l'utilisateur.
- `int()` : convertit en **nombre** pour pouvoir calculer.

Mémo : les mots à retenir

- `print()` : le programme **parle** à l'utilisateur.
- `input()` : le programme **écoute** l'utilisateur (donne du texte).
- `int()` : convertit du texte en **nombre entier** pour pouvoir calculer.
- `if / elif / else` : prendre des décisions.
- `for` : répéter un nombre de fois connu. `while` : répéter tant qu'une condition est vraie.
- `[ ]` liste, `{ }` dictionnaire, `def` fonction.
- `try / except` : éviter qu'une erreur fasse planter le programme.
- `len()` compte les éléments, `append()` ajoute à une liste.